

HW #4.3 QMD File

STAT 184

Mark Zecca

2026-04-11

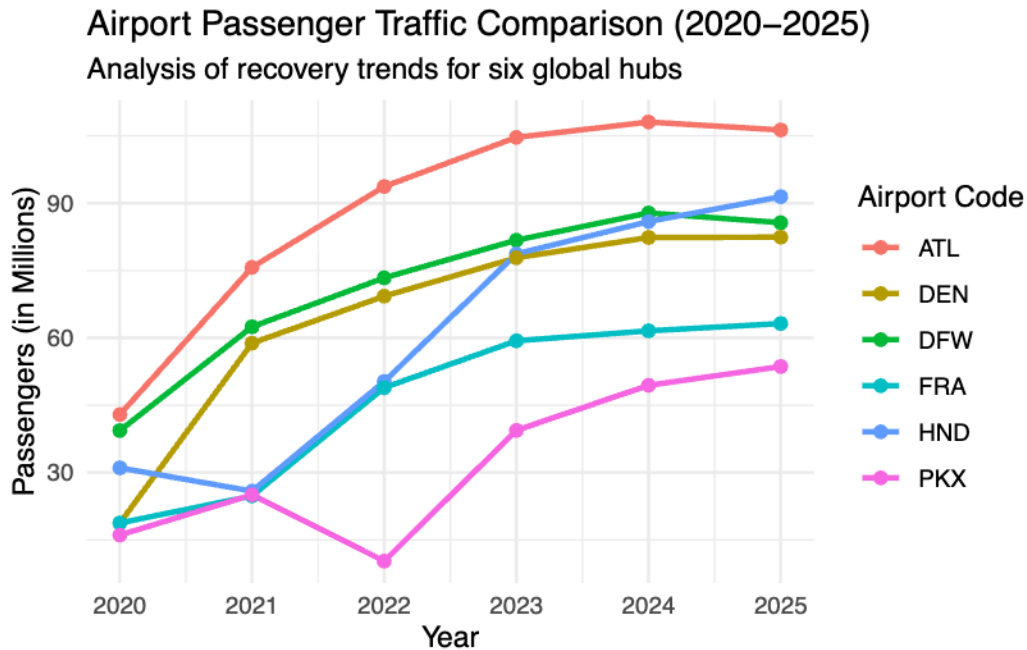
Table of contents

1	Busiest Airports Analysis	2
2	Monte Carlo Numerical Integration	3
3	Planning and Prompting GenAI Tools	4
3.1	My Plan	4
3.2	GenAI Comparison	4
3.2.1	Plan-Based Results	4
3.2.2	Generic Prompt Results	5
3.3	Narrative Comparison	6
4	Reflection	6
5	Appendix 1: GenAI Usage	8
6	Appendix 2: Code Appendix	9

1 Busiest Airports Analysis

The following table and plot illustrate the passenger traffic for six major global airports from 2020 through 2025.

Airport	2025	2024	2023	2022	2021	2020
ATL	106302208	108067766	104653451	93699630	75704760	42918685
DFW	85660127	87817864	81755538	73362946	62465756	39364990
HND	91430000	85900167	78719302	50290705	25876429	31055210
DEN	82427962	82358744	77837917	69286461	58828552	18768601
FRA	63189666	61564957	59355389	48918482	24812849	18770998
PKX	53618949	49441029	39410776	10277623	25051012	16091449



This graph shows how six major airports bounced back between 2020 and 2025. **Atlanta (ATL)** stayed at the top the whole time, hitting a massive **108.1 million** passengers in 2024. Other US airports like **Dallas (DFW)** and **Denver (DEN)** had a pretty steady climb back up, both landing in that **82–85 million** range by 2025. On the flip side, the international spots were all over the place. **Beijing (PKX)** decreased to just **10.3 million** in 2022, while **Tokyo (HND)** had a huge late comeback to **91.4 million**, actually passing DFW and DEN at the very end. Basically, everyone is getting back to normal, but the regional recovery looked a lot different depending on how strict the local travel rules were.

2 Monte Carlo Numerical Integration

The goal of this section is to estimate the integral of a **Standard Normal Distribution (Gaussian)** between $x = -4$ and $x = 4$. Random points are generated across the graph, and the proportion that falls under the function is used to calculate the integral. As the number of points increases, the visualization clearly defines the shape of the bell curve.

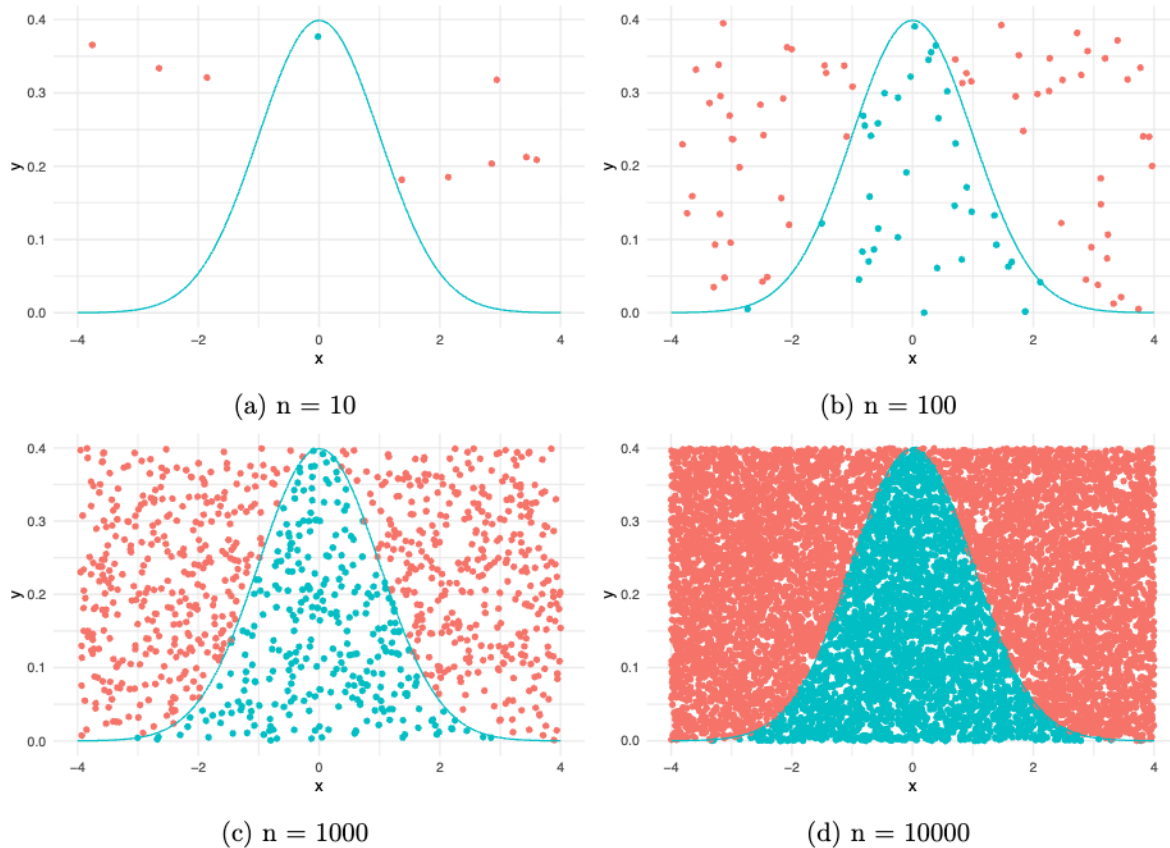


Figure 1: Monte Carlo Numerical Integration

This visualization demonstrates the **Law of Large Numbers**. At $n = 10$, the points are too scattered for a reliable estimate. By $n = 10,000$, the teal region fills the space under the curve, and the estimate converges to **1.0**. This confirms that a larger sample size stabilizes the estimate toward the true value.

3 Planning and Prompting GenAI Tools

3.1 My Plan

The **goal** is to tidy the `calcium.csv` dataset and create a line graph that compares how calcium levels changed over 4 years for the two different groups.

Nouns: `calcium_data`, `group_A`, `group_B`, `clean_data`, `Treatment_Type`, `Year`, `Calcium_Level` **Verbs:** `read_csv()`, `pivot_longer()`, `mutate()`, `bind_rows()`, `ggplot()`

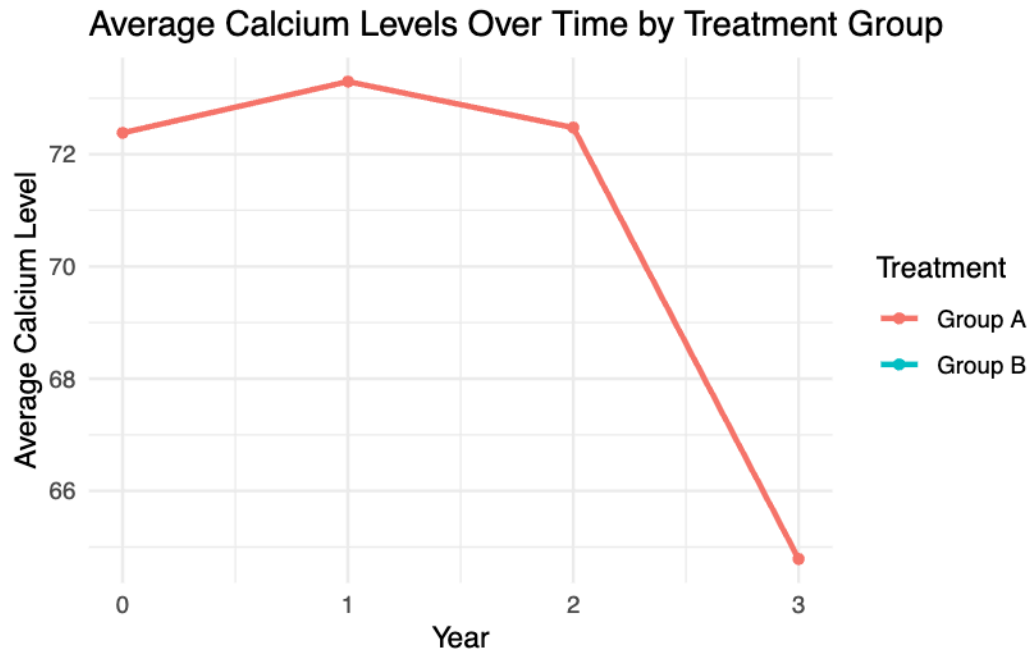
Steps: 1. Load the file and separate it. The first 4 columns are one group. The last 4 columns are the other group. 2. Use `pivot_longer()` to turn those 8 columns into two simple columns. One for the **Year** and one for the **Calcium Level**. 3. Use `mutate()` to tag each row so we know which treatment group the woman belonged to. 4. Combine the two rows back together into one clean table using `bind_rows()`. 5. Use `ggplot()` to put **Year** on the x-axis and **Calcium** on the y-axis, with different colored lines for each group.

3.2 GenAI Comparison

3.2.1 Plan-Based Results

Prompt Used: Columns 1-4 are year measurements (y0, y1, y2, y3) for a group A. Columns 5-8 are measurements (y0.1, y1.1, y2.1, y3.1) for a group B. Following my plan, please write R code to: Import the data. Use `pivot_longer()` to turn the year columns into a single 'Year' column and a 'Calcium_Measure' column. Use `mutate()` to add a 'Treatment' label to each group. Combine both groups into one tidy data frame using `bind_rows()`. Create a `ggplot2` line plot showing the average calcium level over time for both groups, using color to distinguish the treatments.

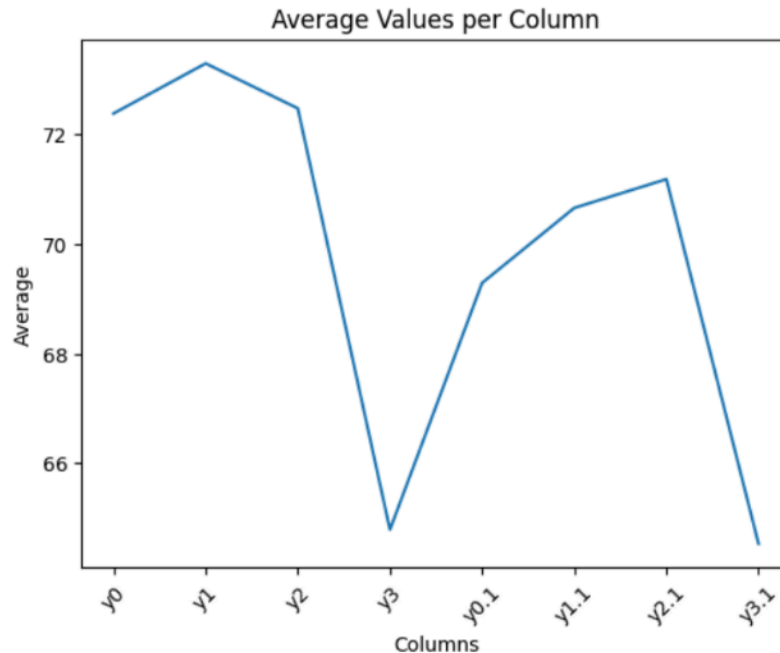
R Output Received:



3.2.2 Generic Prompt Results

Prompt Used: Create a data visualization for these data. Calcium.csv

Result: The generic prompt resulted in a simple average-per-column line chart. The model connected all eight columns into a single line.



3.3 Narrative Comparison

The difference between these results proves that GenAI does much better with a structured plan to handle data effectively. Without context, the generic prompt resulted in a failed visualization that misinterpreted the table structure, connecting all eight columns into a single jagged line with no analytical value. The plan-based prompt utilized a tidy data workflow to correctly separate the experimental groups. By directing the AI to use specific functions like `pivot_longer` and `bind_rows`, the output accurately distinguishes between Group A and Group B. This demonstrates that a structured set of directions can be the difference between a broken output and a reliable analysis.

4 Reflection

Reflecting on the course as a whole, the most significant takeaway has been the shift from simply writing R syntax to managing a complete data analysis workflow. Early in the semester, I focused on basic functional programming, such as building the `roll_dice` and `restaurant_picker` functions to understand how R handles logic, sampling, and basic code structure. I now realize that successful analysis requires a deep understanding of data manipulation, and specifically the importance of planning before any code is written. Establishing a clear plan at the beginning of a project, identifying nouns and verbs, and listing out steps is the only way to ensure a productive analysis. This strategic planning allowed me to identify structural flaws in raw data,

such as recognizing that `calcium.csv` and US Armed Forces datasets were in “wide” formats that required tidy data transformations using `pivot_longer`.

The repetition of creating multiple scatterplots throughout the semester, from diamond prices to pirate beard lengths, helped me build the technical skills needed for more complex tasks. This constant practice with `ggplot2` and aesthetic mapping became the base for the advanced work I completed later in the semester, like Monte Carlo numerical integrations. That project showed that without a clear plan, simulations cannot function. By refining these visualization and wrangling skills through consistent repetition, I became better equipped to handle the structural flaws in raw data.

Ultimately, today’s work with the `calcium.csv` dataset served as a great culmination of skills that I’ve developed this semester. While a generic prompt failed to interpret the table structure, my specific plan involving detailed nouns and verbs guided the AI to an accurate result. This semester has taught me that the true value of analytical work is in the ability to build clear plans and tasklists. This ability to plan and revise is the most vital skill I will take from this course into my future!

5 Appendix 1: GenAI Usage

Tool Used: ChatGPT (Version: GPT-4o)

Date of Usage: April 12, 2026

Exact Prompt:

My Plan The goal is to tidy the calcium.csv dataset and create a make a line graph that compares how calcium levels changed over 4 years for the two different groups. Nouns: calcium_data, group_A, group_B, clean_data, Treatment_Type, Year, Calcium_Level Verbs: read_csv(), pivot_longer(), mutate(), bind_rows(), ggplot() Steps: 1. Load the file and separate it. The first 4 columns are one group. The last 4 columns are the other group. 2. Use pivot_longer() to turn those 8 columns into two simple columns. One for the Year and one for the Calcium Level. 3. Use mutate() to tag each row so we know which treatment group the woman belonged to. 4. Combine the two rows back together into one clean table using bind_rows(). 5. Use ggplot() to put Year on the x-axis and Calcium on the y-axis, with different colored lines for each group. "I have an untidy dataset in the attached calcium.csv file. Here is how the file is structured: Columns 1-4 are year measurements (y0, y1, y2, y3) for Group A. Columns 5-8 are measurements (y0.1, y1.1, y2.1, y3.1) for Group B. Following my plan, please write R code to: Import the data. Use pivot_longer() to turn the year columns into a single 'Year' column and a 'Calcium_Measure' column. Use mutate() to add a 'Treatment' label to each group. Combine both groups into one tidy data frame using bind_rows(). Create a ggplot2 line plot showing the average calcium level over time for both groups, using color to distinguish the treatments.

Exact Response Text:

```
1.Import the data calcium_data <- read_csv("calcium.csv")

2 Separate and reshape Group A group_A <- calcium_data %>% select(y0, y1, y2, y3) %>%
pivot_longer( cols = everything(), names_to = "Year", values_to = "Calcium_Measure" )
%>% mutate( Year = as.numeric(str_remove(Year, "y")), Treatment = "Group A" )

3.Separate and reshape Group B group_B <- calcium_data %>% select(y0.1, y1.1, y2.1, y3.1)
%>% pivot_longer( cols = everything(), names_to = "Year", values_to = "Calcium_Measure"
) %>% mutate( Year = as.numeric(str_remove(Year, "y\\.|\\.1")), Treatment = "Group B" )

4.Combine into one tidy dataset clean_data <- bind_rows(group_A, group_B)

5.Calculate averages summary_data <- clean_data %>% group_by(Treatment, Year) %>%
summarise( avg_calcium = mean(Calcium_Measure, na.rm = TRUE), .groups = "drop" )

6.Plot ggplot(summary_data, aes(x = Year, y = avg_calcium, color = Treatment)) +
geom_line(size = 1) + geom_point() + labs( title = "Average Calcium Levels Over Time", x
= "Year", y = "Average Calcium Level" ) + theme_minimal()
```


6 Appendix 2: Code Appendix

```
library(tidyr)

airport_base <- data.frame(
  Airport = c("ATL", "DFW", "HND", "DEN", "FRA", "PKX"),
  `2025` = c(106302208, 85660127, 91430000, 82427962, 63189666, 53618949),
  `2024` = c(108067766, 87817864, 85900167, 82358744, 61564957, 49441029),
  `2023` = c(104653451, 81755538, 78719302, 77837917, 59355389, 39410776),
  `2022` = c(93699630, 73362946, 50290705, 69286461, 48918482, 10277623),
  `2021` = c(75704760, 62465756, 25876429, 58828552, 24812849, 25051012),
  `2020` = c(42918685, 39364990, 31055210, 18768601, 18770998, 16091449),
  check.names = FALSE
)

airport_tidy <- pivot_longer(
  data = airport_base,
  cols = `2025`:`2020`,
  names_to = "Year",
  values_to = "Total_Passengers"
)

airport_tidy$Year <- as.numeric(airport_tidy$Year)

library(tidyr)

# Pivot the data back to wide format
airport_table_wide <- pivot_wider(
  data = airport_tidy,
  names_from = Year,
  values_from = Total_Passengers
)

# Open the table in a separate, pop-out window
knitr::kable(airport_table_wide)

library(ggplot2)
```

```

ggplot(data = airport_tidy, aes(x = Year, y = Total_Passengers / 1e6, color = Airport)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 2020:2025) +
  theme_minimal() +
  labs(
    title = "Airport Passenger Traffic Comparison (2020-2025)",
    subtitle = "Analysis of recovery trends for six global hubs",
    x = "Year",
    y = "Passengers (in Millions)",
    color = "Airport Code"
  )
library(tidyverse)

generate_mc_points <- function(n, x_min, x_max, y_min, y_max) {
  x_coords <- runif(n, min = x_min, max = x_max)
  y_coords <- runif(n, min = y_min, max = y_max)
  points_df <- data.frame(x = x_coords, y = y_coords, sample_size = n)
  return(points_df)
}

mc_10 <- generate_mc_points(10, -4, 4, 0, 0.4) %>%
  mutate(f_x = dnorm(x, mean = 0, sd = 1), flag = ifelse(y <= f_x, "on/below", "above"))
ggplot(mc_10, aes(x = x, y = y, color = flag)) + geom_point() +
  stat_function(fun = dnorm, args = list(mean = 0, sd = 1), xlim = c(-4, 4)) +
  theme_minimal() + theme(legend.position = "none")

mc_100 <- generate_mc_points(100, -4, 4, 0, 0.4) %>%
  mutate(f_x = dnorm(x, mean = 0, sd = 1), flag = ifelse(y <= f_x, "on/below", "above"))
ggplot(mc_100, aes(x = x, y = y, color = flag)) + geom_point() +
  stat_function(fun = dnorm, args = list(mean = 0, sd = 1), xlim = c(-4, 4)) +
  theme_minimal() + theme(legend.position = "none")

mc_1000 <- generate_mc_points(1000, -4, 4, 0, 0.4) %>%
  mutate(f_x = dnorm(x, mean = 0, sd = 1), flag = ifelse(y <= f_x, "on/below", "above"))
ggplot(mc_1000, aes(x = x, y = y, color = flag)) + geom_point() +
  stat_function(fun = dnorm, args = list(mean = 0, sd = 1), xlim = c(-4, 4)) +
  theme_minimal() + theme(legend.position = "none")

mc_10000 <- generate_mc_points(10000, -4, 4, 0, 0.4) %>%
  mutate(f_x = dnorm(x, mean = 0, sd = 1), flag = ifelse(y <= f_x, "on/below", "above"))

```

```

ggplot(mc_10000, aes(x = x, y = y, color = flag)) + geom_point() +
  stat_function(fun = dnorm, args = list(mean = 0, sd = 1), xlim = c(-4, 4)) +
  theme_minimal() + theme(legend.position = "none")
library(tidyverse)

# 1. Import the data
calcium_data <- read_csv("calcium.csv", trim_ws = TRUE)
names(calcium_data) <- c("y0", "y1", "y2", "y3", "y0.1", "y1.1", "y2.1", "y3.1")

# 2. Separate groups
group_A <- calcium_data %>%
  select(y0, y1, y2, y3) %>%
  pivot_longer(
    cols = everything(),
    names_to = "Year",
    values_to = "Calcium_Measure"
  ) %>%
  mutate(
    Year = as.numeric(str_remove(Year, "y")),
    Treatment = "Group A"
  )

group_B <- calcium_data %>%
  select(y0.1, y1.1, y2.1, y3.1) %>%
  pivot_longer(
    cols = everything(),
    names_to = "Year",
    values_to = "Calcium_Measure"
  ) %>%
  mutate(
    Year = as.numeric(str_remove(Year, "y\\.\\.\\.1")),
    Treatment = "Group B"
  )

# 3 & 4. Combine into one tidy dataset
clean_data <- bind_rows(group_A, group_B)

# 5. Create summary (average per year per group)
summary_data <- clean_data %>%
  group_by(Treatment = as.factor(Treatment), Year) %>%
  summarise(
    avg_calcium = mean(Calcium_Measure, na.rm = TRUE),

```

```
    .groups = "drop"
  )

# 6. Plot
ggplot(summary_data, aes(x = Year, y = avg_calcium, color = Treatment, group = Treatment)) +
  geom_line(size = 1) +
  geom_point() +
  labs(
    title = "Average Calcium Levels Over Time by Treatment Group",
    x = "Year",
    y = "Average Calcium Level"
  ) +
  theme_minimal()
```
